Module 13 — Production Scenarios (Debugging)

> Senior interviews lean heavily on “walk me through debugging X in production.” The winning approach is always: **symptom → hypotheses → collect evidence (metrics/logs/traces/dumps) → isolate → fix → verify → prevent.** Below is a playbook per incident with the exact tools.

13.1 The universal debugging framework

```
1. Define the symptom & blast radius (which endpoint/service, since when, % affected)
2. Correlate: dashboards (RED/USE), recent deploys/config changes, traces
3. Form hypotheses (resource? dependency? code? data?)
4. Collect evidence: thread dump, heap dump, GC log, DB stats, slow-query log
5. Isolate the bottleneck (one variable at a time)
6. Mitigate (rollback/scale/flag) then fix root cause
7. Verify with metrics; add alert/test to prevent recurrence
```

Core JVM tools: `jstack` (threads), `jmap` / `jcmd GC.heap_dump` (heap), `jstat` /GC logs (GC), `jcmd`, async-profiler / JFR (CPU + locks + allocation), Actuator (`/threaddump`, `/heapdump`, `/metrics`).

13.2 Memory Leak

- **Symptom:** heap grows over time, Old Gen never shrinks after full GC, eventually `OutOfMemoryError: Java heap space`; rising GC time.
- **Diagnose:** enable `-XX:+HeapDumpOnOutOfMemoryError`; take heap dump (`jmap -dump:live` / Actuator `/heapdump`); open in Eclipse **MAT** → dominator tree + retained size → find the growing object graph and GC roots.
- **Common causes:** unbounded static/`ThreadLocal` collections/caches (no TTL/max size), listeners/connections never removed, classloader leaks (Metaspace), caching entities forever, `ThreadLocal` not cleared in pooled threads.
- **Fix:** bound caches (size+TTL), remove listeners, clear `ThreadLocal`, weak references where appropriate.

13.3 High CPU

- **Symptom:** CPU pegged, latency up.
- **Diagnose:** `top -H -p <pid>` to find hot threads → convert TID to hex → match in `jstack`; or **async-profiler**/ JFR flame graph.
- **Common causes:** infinite/busy loops, excessive GC (see GC log — CPU burned in GC), inefficient algorithm ($O(n^2)$), regex catastrophic backtracking, serialization, chatty logging.
- **Fix:** optimize hot path, tune GC/allocation, add caching, fix the loop.

13.4 Slow APIs

- **Symptom:** high p95/p99 latency.
- **Diagnose:** distributed **trace** to find the slow span (DB? downstream? external?), DB slow-query log + `EXPLAIN`, check GC pauses, pool waits.
- **Common causes:** N+1 queries, missing index, slow/blocking downstream call without timeout, lock contention, connection-pool wait, large payloads/no pagination, cold cache.
- **Fix:** index/rewrite query, fix N+1 (EntityGraph), add cache, timeouts + circuit breaker, paginate, async/parallelize.

13.5 Deadlocks

- **Symptom:** threads hang, throughput drops, no progress.
- **Diagnose:** `jstack` → “Found one Java-level deadlock” section (which threads hold/wait which locks). DB deadlocks: DB logs / `SHOW ENGINE INNODB STATUS`.
- **Common causes:** inconsistent lock ordering, nested `synchronized`, DB rows locked in different order across transactions.
- **Fix:** consistent global lock ordering, `tryLock` with timeout, reduce lock scope, keep transactions short and same-ordered.

13.6 Connection Pool Exhaustion

- **Symptom:** `HikariPool - Connection is not available, request timed out`; latency spikes; requests queue.
- **Diagnose:** HikariCP metrics (active/idle/pending), thread dump (threads blocked getting a connection), find long-running queries / transactions.
- **Common causes:** long transactions, **remote/HTTP calls inside a transaction**, leaked connections (not closed), pool too small, slow queries holding connections.
- **Fix:** shorten transactions, move I/O outside tx, fix leaks (try-with- resources / Spring-managed), right-size pool, add query timeouts, `leakDetectionThreshold`.

13.7 Kafka Consumer Lag

- **Symptom:** lag (latest−committed offset) grows; data freshness degrades.
- **Diagnose:** `kafka-consumer-groups --describe`, lag metrics; check per- partition lag, consumer CPU, processing time, rebalances.
- **Common causes:** slow processing, too few consumers/partitions, blocking calls per message, frequent rebalances, huge messages.
- **Fix:** scale consumers (≤ #partitions), add partitions, batch processing, async/parallel within consumer, tune `max.poll.records`/`max.poll.interval.ms`, move slow work off the poll thread.

13.8 Redis Cache Misses

- **Symptom:** low hit ratio, DB load high, latency up.
- **Diagnose:** Redis `INFO stats` (keyspace hits/misses), `INFO memory`, evicted keys, monitor TTLs.
- **Common causes:** TTL too short, eviction under memory pressure (LRU/LFU evicting hot keys), cache stampede on expiry, penetration (missing keys), key design (per-user vs shared).
- **Fix:** right-size memory + eviction policy, tune TTL + jitter, stampede protection (lock/coalesce), cache negatives, warm cache.

13.9 Database Bottlenecks

- **Symptom:** DB CPU/IO high, slow queries, lock waits.
- **Diagnose:** slow-query log, `EXPLAIN ANALYZE`, `pg_stat_activity`/ `SHOW PROCESSLIST`, lock/wait stats, index usage stats.
- **Common causes:** missing/wrong indexes, N+1, full scans, hot rows/locks, no pagination, unbatched writes, stale statistics.

- **Fix:** indexes, query rewrite, caching, read replicas, batching, partitioning/ sharding for scale, connection pool tuning.

13.10 Thread Starvation

- **Symptom:** requests queue while CPU idle; pool/executor saturated; `RejectedExecutionException`.
- **Diagnose:** thread dump (many threads BLOCKED/WAITING on I/O or locks), pool metrics (active == max, growing queue).
- **Common causes:** blocking I/O on a small/shared pool (Tomcat workers, `ForkJoinPool.commonPool`), no timeouts, bulkhead missing, deadlock/contention.
- **Fix:** bounded dedicated pools + bulkheads, timeouts, non-blocking/async or virtual threads for I/O, don't block the common pool.

13.11 Circular Dependency (startup)

- **Symptom:** app fails to start; `BeanCurrentlyInCreationException` / "circular reference" (or `Requested bean is currently in creation`).
- **Diagnose:** read the cycle Spring prints (A→B→A).
- **Cause/Fix:** constructor cycle can't be resolved; refactor (extract 3rd bean), `@Lazy` on one dependency, or setter injection (Module 2.6). Note Boot 2.6+ forbids circular refs by default.

13.12 Bean Creation Errors (startup)

- **Symptoms & causes:**
- `NoSuchBeanDefinitionException` — dependency not a bean / not scanned → add stereotype/`@Bean`, check `@ComponentScan` packages.
- `NoUniqueBeanDefinitionException` — multiple candidates → `@Primary`/`@Qualifier`.
- `UnsatisfiedDependencyException` — wrapping the real cause (read the "Caused by" chain — often a property/ DB connection failure).
- `BeanCreationException` — often config/property/DB issues at init.
- **Fix:** read the **root cause chain** bottom-up; verify component scan, profiles active, required properties present, auto-config conditions (`--debug`).

Module 13 — One-Page Cheat Sheet (symptom → tool → cause)

Incident	First tool	Usual root cause
Memory leak	heap dump + MAT	unbounded cache/collection/ThreadLocal
High CPU	top -H + jstack / profiler	busy loop, GC thrash, bad algo
Slow API	distributed trace + EXPLAIN	N+1, missing index, no timeout
Deadlock	jstack	inconsistent lock ordering
Pool exhaustion	Hikari metrics + jstack	tx holding conn during remote call
Kafka lag	consumer-groups –describe	slow processing / too few consumers
Cache miss	Redis INFO stats	TTL/eviction/stampede
DB bottleneck	slow log + EXPLAIN	missing index / N+1 / locks
Thread starvation	thread dump + pool metrics	blocking I/O on small pool
Circular dep	startup log	constructor cycle
Bean errors	root-cause chain	missing/ambiguous bean, bad prop

Module 13 — Top Interview Questions

1. How do you debug a memory leak in production? (heap dump → MAT → dominators)
2. High CPU — how do you find the hot thread? (top -H → jstack/profiler)
3. Latency spike — how do you localize it? (trace → span → EXPLAIN)
4. How do you detect a deadlock? (jstack “Found one Java-level deadlock”)
5. Connection pool exhaustion — causes and fix.
6. Growing Kafka lag — diagnose and remediate.
7. Low cache hit ratio — what do you check?
8. Thread starvation vs deadlock — how to tell apart.
9. App won’t start with a bean error — how do you read it?
10. What data do you always capture before restarting a sick JVM? (thread + heap dump, GC log)

Module 13 — Common Mistakes

- Restarting before capturing dumps (losing evidence).
- Blaming CPU when it’s GC or lock contention.
- Ignoring recent deploys/config changes as the trigger.
- Fixing symptoms (bigger pool) without root cause.

Module 13 — Mock Interview

1. *“Service memory climbs until OOM every few days.”* → likely leak; heap-dump-on-OOM, MAT dominator tree; find unbounded cache; bound size+TTL.
2. *“CPU at 100%, latency up, no traffic spike.”* → `top -H` hot thread → `jstack` → busy loop / GC log shows GC thrash; fix code or allocation.
3. *“Half of* `/checkout` *calls time out at 30s.”* → connection-pool timeout; jstack shows threads waiting for connections; a transaction is calling an external API — move it out, add timeout.
4. *“Kafka lag on one partition only.”* → hot key / uneven partitioning or a slow message; check per-partition processing time; rebalance keys / DLQ poison messages.

5. *“App won’t boot: UnsatisfiedDependencyException.”* → read the Caused-by chain to the bottom — often a missing property or DB connection; fix config, verify profile.

Next → Module 14: Frequently Asked Coding Questions.